09

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

HTTP 메서드와 상태 코드

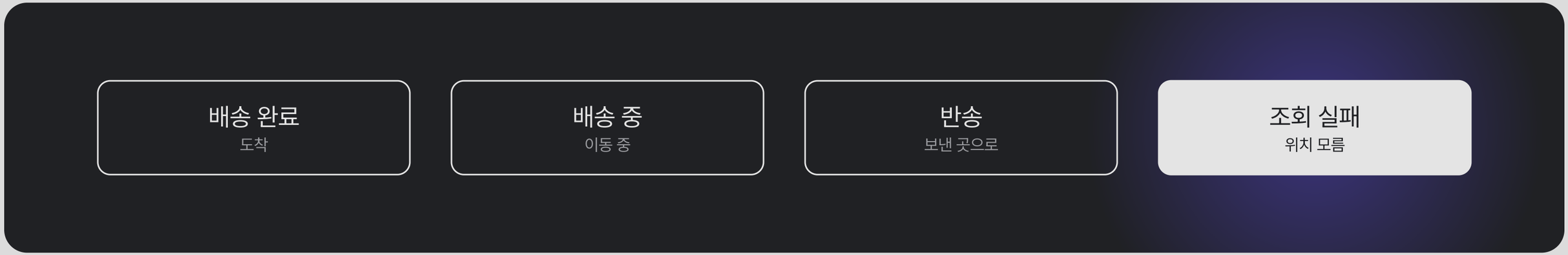
제1부 기반: 백엔드와 Node.js 런타임 · 3장 HTTP 기초

숫자 세 자리를 읽는 쪽은 사람이 아닌 프로그램



© 한상훈 NestJS 백엔드 마스터

택배 조회 화면



→ 배송 완료, 배송 중, 반송

→ 조회 실패: 택배 위치를 모른다는 뜻, 미배송과 다름

→ 조회 실패를 미배송으로 읽고 재주문 → 물건 두 개

© 한상훈 NestJS 백엔드 마스터

응답의 첫 줄

응답

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 34

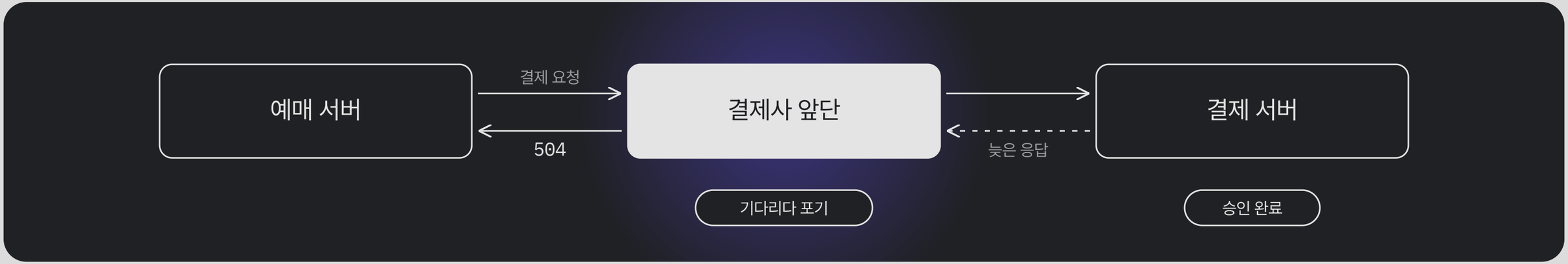
{"id":1,"title":"10월 콘서트"}
```

→ 상태 코드(status code):
응답 첫 줄의 숫자 세 자리

→ 200 성공, 404 없음, 500 서버 내부 오류

© 한상훈 NestJS 백엔드 마스터

결제사가 준 504



→ 예매 서버 → 결제사 앞단 → 결제 서버

→ 504: 앞단이 결제 서버를 기다리다 포기

→ 결제 여부: 이 숫자로는 알 수 없음

→ '결제 실패, 다시 결제' 안내 → 이미 결제된 손님의 두 번째 결제

→ 흔한 앱 코드: 200이 아니면 전부 실패 처리

© 한상훈 NestJS 백엔드 마스터

요청의 첫 줄

요청

```
POST /performances/1/holds HTTP/1.1
Host: localhost:4009
Content-Type: application/json

{"seat":"A-3-15"}
```

→ 메서드(method): 이 주소에 하려는 일, 요청의 동사

→ POST + /performances/1/holds: 1번 회차의 좌석 선점 요청

© 한상훈 NestJS 백엔드 마스터

메서드 일곱 개

메서드	뜻	본문
GET	조회	응답에 있음
POST	새로 만들기, 처리 맡기기	요청에 있음
PUT	통째로 이 값으로	요청에 있음
PATCH	일부만 수정	요청에 있음
DELETE	삭제	보통 없음
HEAD	GET과 같되 헤더만	응답에 없음
OPTIONS	받는 메서드 문의	보통 없음

© 한상훈 NestJS 백엔드 마스터

예매 플랫폼의 메서드

요청	하는 일
`GET /concerts/1`	1번 공연 조회
`POST /performances/1/holds`	좌석 선점
`DELETE /performances/1/holds/1`	1번 선점 해제
`PATCH /concerts/1`	공연 정보 일부 수정

© 한상훈 NestJS 백엔드 마스터

동사를 주소에 넣은 API

요청	캐시	라이브러리 재시도
`POST /getConcert`	저장 안 함	안 함
`GET /concerts/1`	저장 가능	함

→ 주소: 대상, 메서드: 할 일

→ 모두 POST로 통일한 API: 메서드가 알려 주던 정보 소실

© 한상훈 NestJS 백엔드 마스터

Nest 데코레이터와 라우트

holds.controller.ts

```ts
@Controller('performances/:performanceId')
export class HoldsController {
  @Post('holds')
  hold(@Param('performanceId') performanceId: string, @Body('seat') seat: string) {
    return this.concertsService.hold(+performanceId, seat);
  }
```

앱 시작 로그

```
LOG [RouterExplorer] Mapped {/performances/:performanceId/holds, POST} route
LOG [RouterExplorer] Mapped {/performances/:performanceId/holds/:holdId, DELETE} route
```

→ 좌석: 회차(공연의 날짜별 회)마다 따로 판매, 회차 주소 아래에

→ @Get, @Post, @Put, @Patch, @Delete, @Head, @Options

© 한상훈 NestJS 백엔드 마스터

없는 메서드, HEAD, OPTIONS

터미널

```
$ curl -i -X PUT localhost:4009/concerts/1
HTTP/1.1 404 Not Found
{"message":"Cannot PUT /concerts/1","error":"Not Found","statusCode":404}

$ curl -I localhost:4009/concerts/1
HTTP/1.1 200 OK
Content-Length: 34

$ curl -i -X OPTIONS localhost:4009/concerts/1
HTTP/1.1 404 Not Found
```

→ 주소는 있고 메서드만 없음: 표준은 405, Express 라우터는 404

→ HEAD: GET 라우트가 자동 처리, 본문 없이 길이만

데이터 블록: NestJS 12.2.0, Express 5.3.0 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

안전한 메서드, 멱등한 메서드

메서드	안전함	멱등함
GET, HEAD, OPTIONS	O	O
PUT, DELETE	X	O
POST, PATCH	X	X

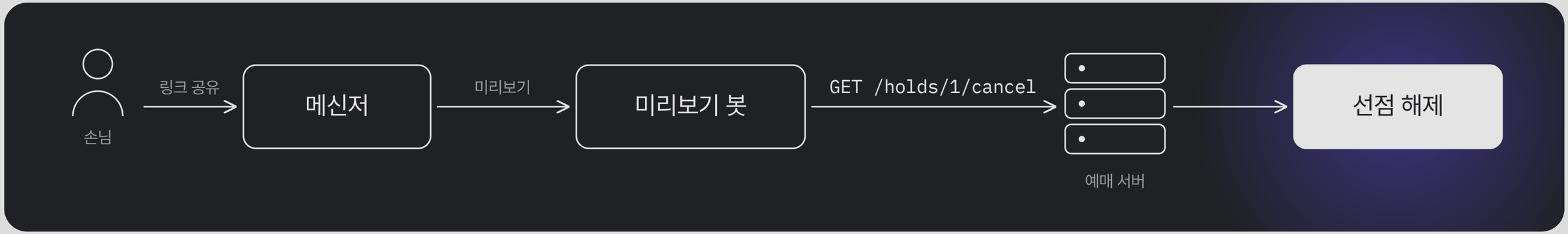
→ 안전함(safe): 서버 상태를 바꾸지 않겠다는 약속

→ 멱등함(idempotent): 한 번 보내든 열 번 보내든 서버 상태가 같음

→ 근거: RFC 9110 9.2절

© 한상훈 NestJS 백엔드 마스터

선점을 푸는 GET 링크



→ 가정: 선점 해제를 `GET /holds/1/cancel` 링크로 구현

→ 손님이 메신저로 링크 공유

→ 메신저의 링크 미리보기가 그 주소로 GET 요청

→ 공유한 순간 선점 해제

→ 같은 일: 검색 엔진 크롤러, 브라우저 미리 가져오기

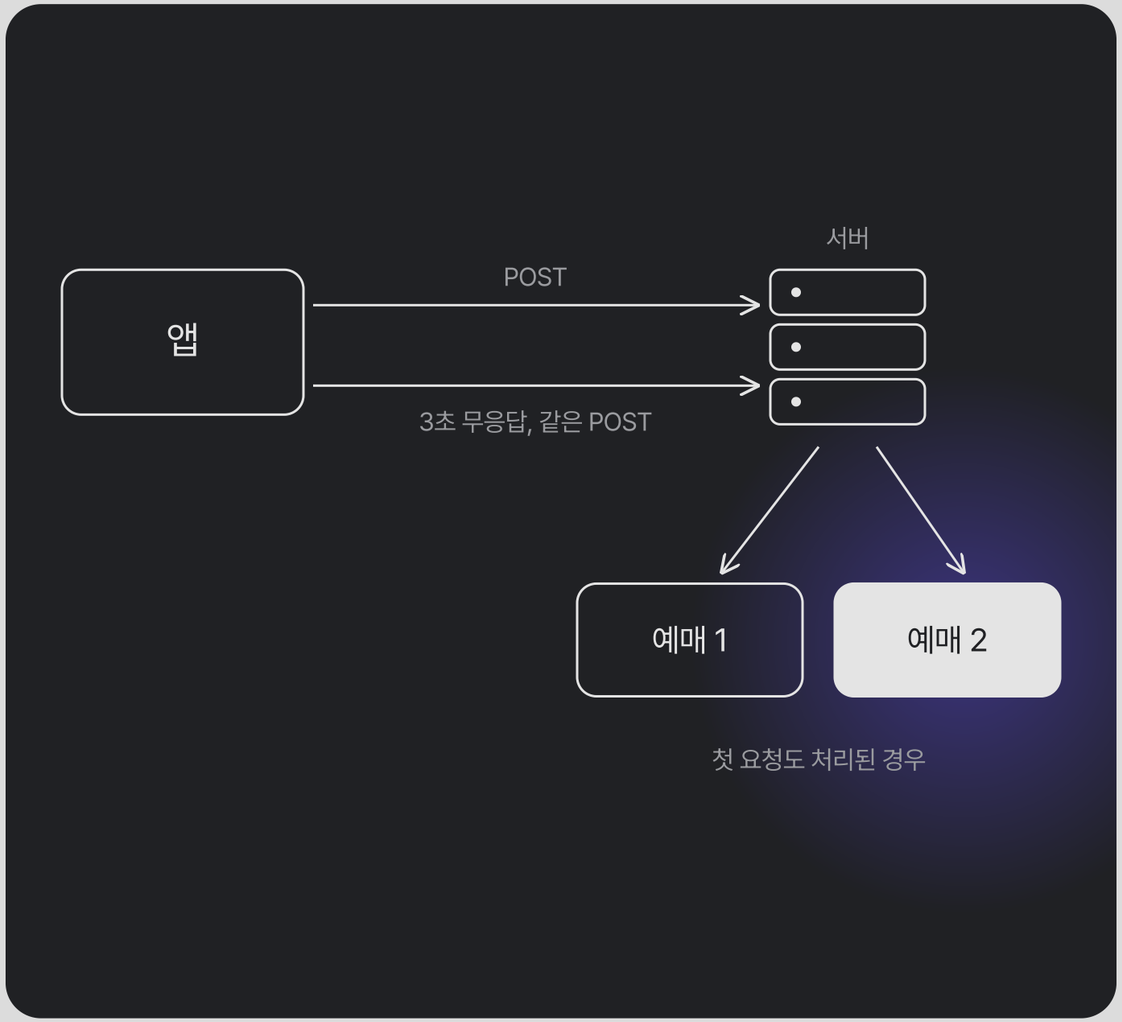
© 한상훈 NestJS 백엔드 마스터

다시 보낸 POST

→ 가정: 예매 요청 POST, 3초 동안 응답 없음 → 같은 요청 다시 전송

→ 첫 요청이 서버에서 처리됐다면 예매 두 건

→ 브라우저 새로고침의 '양식 다시 제출' 확인 창: 같은 이유



© 한상훈 NestJS 백엔드 마스터

undici 재시도 기본 메서드

```
// node_modules/undici/lib/handler/retry-handler.js
methods: methods ?? ['GET', 'HEAD', 'OPTIONS', 'PUT', 'DELETE', 'TRACE'],
```

→ undici: Node의 `fetch`를 만든 HTTP 클라이언트

→ 기본 재시도 대상: 멱등한 메서드만, POST·PATCH 제외

데이터 블록: undici 6.20.1 소스 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

PUT, PATCH, DELETE를 두 번

요청	두 번 보낸 결과
`PUT /seats/A-3-15 {"status":"held"}`	같음: held
`PATCH /concerts/1 {"views":"+1"}` 같은 증가	다름: 두 번 증가
`DELETE /performances/1/holds/1`	응답 `204` → `404`, 서버 상태는 같음

© 한상훈 NestJS 백엔드 마스터

상태 코드 다섯 묶음

묶음	뜻	예
1xx	처리 중 정보	101 프로토콜 전환
2xx	성공	200, 201, 204
3xx	다른 곳 안내, 캐시 사용	301, 302, 304
4xx	요청한 쪽 문제	400, 401, 403, 404, 409, 429
5xx	서버 쪽 문제	500, 502, 503, 504

© 한상훈 NestJS 백엔드 마스터

한 번 준 301

Moved Permanently

301

영구 이동, 브라우저가 기억

Found

302

임시 이동

→ 가정: 예매 페이지 주소 변경에 301 → 하루 뒤 원복

→ 301을 받은 브라우저: 서버에 묻지 않고 계속 새 주소로

→ 되돌릴 수 있는 이동: 302

© 한상훈 NestJS 백엔드 마스터

4xx와 5xx, 다음 동작

받은 응답	같은 요청 재전송	받는 쪽 할 일
4xx	결과 같음	요청 수정, 손님 안내
5xx	결과가 바뀔 수 있음	잠시 뒤 재시도, 장애 알림

→ 모니터링: 5xx 비율로 장애 알림

© 한상훈 NestJS 백엔드 마스터

모든 응답이 200인 API

응답

```
HTTP/1.1 200 OK
{"success":false,"error":"결제사 연결 실패"}
```

→ 가정: 결제가 전부 실패하는 상황

→ 에러율 대시보드: 0%, 장애 알림 없음

→ 앞단 캐시: 실패 응답을 성공 응답으로 저장

→ 재시도 기능: 200이므로 재시도 없음

→ 먼저 울리는 것: 고객센터 전화

© 한상훈 NestJS 백엔드 마스터

45만 명의 매진 응답

매진 응답	오픈 1초의 대시보드
`500`	에러율 90%, 알림 폭주, 진짜 장애가 섞여 안 보임
`200` + 본문 '매진'	매진 건수를 상태 코드로 셀 수 없음
`4xx`	5xx는 진짜 장애만, 매진은 4xx로 따로 집계

→ 좌석 5만 석, 접속 50만 명 (가정값)

© 한상훈 NestJS 백엔드 마스터

400, 404, 409

코드	뜻	예매 플랫폼 예
`400 Bad Request`	요청 모양이 틀림	좌석 번호 자리에 숫자 15
`404 Not Found`	대상이 없음	99번 공연
`409 Conflict`	지금 서버 상태와 충돌	이미 선점된 `A-3-15`

→ 검증 실패: `400` 또는 `422`, API 안에서 하나로 (Nest 검증 기본값 `400`)

→ `400`을 받은 앱: 손님 안내가 아닌 앱 코드 버그

© 한상훈 NestJS 백엔드 마스터

401과 403

코드	실제 뜻	받는 쪽 할 일
`401 Unauthorized`	누군지 모름 (로그인 안 함, 토큰 만료)	다시 로그인
`403 Forbidden`	누군지 알지만 권한 없음	안내, 로그인해도 같음

→ 가정: `401`이면 로그아웃시키는 앱, 권한 없는 화면에 `401` 응답

→ 결과: 로그인 → 같은 화면 → 다시 로그인 화면, 반복

© 한상훈 NestJS 백엔드 마스터

있다는 걸 숨기는 404

→ GitHub API: 권한 없는 비공개 저장소 요청에 403 대신 404

→ 이유: 403은 '그 저장소가 있다'는 정보 노출

→ 남의 예매 내역 조회: 403 대신 404도 선택지

출처: GitHub REST API 문서 (부록 시트)

© 한상훈 NestJS 백엔드 마스터

429와 Retry-After

응답

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
```

→ 짧은 시간에 너무 많은 요청

→ `Retry-After`: 다시 보내도 되는 시점(초 또는 날짜)

→ 받는 쪽: 그 시간까지 대기 후 재전송

© 한상훈 NestJS 백엔드 마스터

Nest의 기본 상태 코드

holds.controller.ts

```
@Delete('holds/:holdId')
@HttpCode(204)
release(@Param('performanceId') performanceId: string, @Param('holdId') holdId: string) {
  this.concertsService.release(+performanceId, +holdId);
}
```

→ 기본값: POST 201 Created, 나머지 200 OK

→ 201 응답의 Location 헤더: 새로 만든 것의 주소 (예: /performances/1/holds/1)

→ 다른 숫자: @HttpCode

→ 실패: 예외 클래스로 (NotFoundException → 404, ConflictException → 409)

© 한상훈 NestJS 백엔드 마스터

좌석 선점 껍데기

concerts.service.ts

```
findOne(id: number) {
  const concert = this.concerts.find((c) => c.id === id);
  if (!concert) {
    throw new NotFoundException(`공연 ${id}번이 없습니다`);
  }
  return concert;
}

hold(performanceId: number, seat: string) {
  const key = `${performanceId}:${seat}`;
  if (this.holds.has(key)) {
    throw new ConflictException(`Seat ${seat} is already held`);
  }
  const hold = { holdId: this.nextHoldId++, performanceId, seat };
  this.holds.set(key, hold);
  return hold;
}
```

→ 저장: 메모리의 Map, DB 없음

© 한상훈 NestJS 백엔드 마스터

200, 404, 201, 409, 204

터미널

```
$ curl -i localhost:4009/concerts/1
HTTP/1.1 200 OK
{"id":1,"title":"10월 콘서트"}
$ curl -i localhost:4009/concerts/99
HTTP/1.1 404 Not Found
{"message":"공연 99번이 없습니다","error":"Not Found","statusCode":404}
$ curl -i -X POST localhost:4009/performances/1/holds -d '{"seat":"A-3-15"}' ...
HTTP/1.1 201 Created
{"holdId":1,"performanceId":1,"seat":"A-3-15"}
$ (같은 요청 한 번 더)
HTTP/1.1 409 Conflict
{"message":"Seat A-3-15 is already held","error":"Conflict","statusCode":409}
$ curl -i -X DELETE localhost:4009/performances/1/holds/1
HTTP/1.1 204 No Content
```

© 한상훈 NestJS 백엔드 마스터

처리하지 못한 예외

concerts.controller.ts

```ts
@Get(':id/sales')
sales(@Param('id') id: string) {
  throw new Error('settlement server did not respond');
}
```

→ 응답: 일반 문장만

→ 원래 메시지와 스택: 서버 로그에만

○ ○ ○ 터미널

```
$ curl -i localhost:4009/concerts/1/sales
HTTP/1.1 500 Internal Server Error
{"statusCode":500,"message":"Internal server error"}

(서버 로그)
ERROR [ExceptionsHandler] Error: settlement server did not respond
    at ConcertsController.sales (.../main.js:227:15)
```

© 한상훈 NestJS 백엔드 마스터

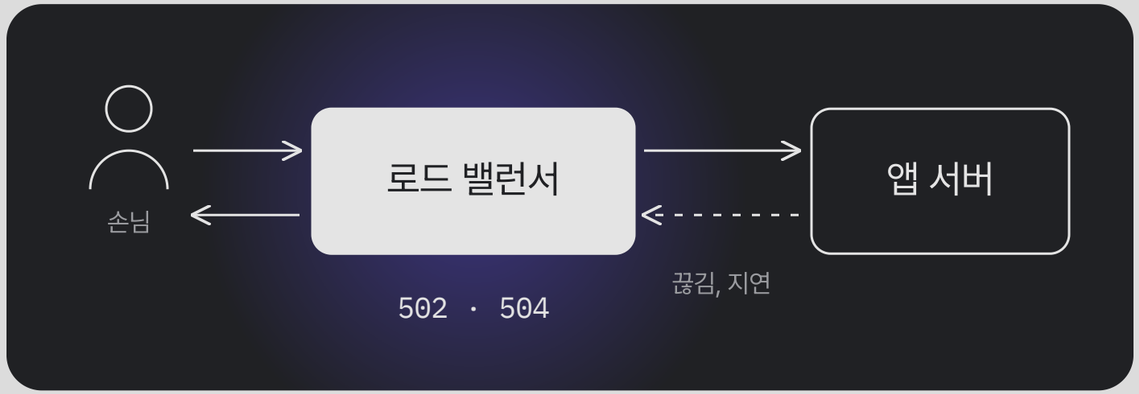
5xx 넷

코드	뜻	누가 만드나
500 Internal Server Error	앱 안에서 처리 못 한 예외	앱
502 Bad Gateway	뒤 서버의 응답이 이상하거나 연결이 끊김	앞단
503 Service Unavailable	지금은 못 받음 (점검, 과부하)	앱 또는 앞단
504 Gateway Timeout	뒤 서버를 기다리다 포기	앞단

→ 앞단: 로드 밸런서, 리버스 프록시, API 게이트웨이

© 한상훈 NestJS 백엔드 마스터

앞단이 만드는 502, 504



→ 손님 → 로드 밸런서 → 앱 서버

→ 502·504 응답: 로드 밸런서가 생성

→ 앱 로그: 그 요청 기록이 없을 수 있음

→ 확인할 곳: 앞단의 접근 로그

받은 응답	받는 쪽 코드
4xx	재전송 없이 손님 안내
429, 503 + Retry-After	그 시간만큼 대기 후 재전송
502, 504	결과 모름, 조회 먼저
500	GET은 한 번 재전송 가능, POST는 조회 먼저

© 한상훈 NestJS 백엔드 마스터

결과를 모르는 504

모양 1

답 없음

모양 2

느림

504일 가능성

성공 후 응답 유실

→ 결제 호출이 실패하는 세 모양: 답 없음, 느림, 성공 후 응답 유실

→ 504: 세 번째일 가능성

→ 실패로 확정 금지 → '확인 중' 상태

→ 결제사에 결과 재조회 후 확정

→ 택배 조회 실패 → 재주문 대신 재조회

© 한상훈 NestJS 백엔드 마스터

다음 동작을 정하는 숫자

상태 코드는 응답을 받은 프로그램이
다음에 할 일을 정하는 숫자